Color Space Conversion Progress Report

1. Brendon Waters - V00963713 - brendonwaters@uvic.ca
2. Matt Smith - V00966014 - matthewsmith@uvic.ca

Table of Contents

Introduction

Our project is to perform efficient color conversion of up to approximately 320 by 180 pixels from the RGB color space to YCbCr (Luminance, Chroma Blue, Chroma Red) and back with minimal data loss. We expect to implement our code using only fixed-point arithmetic and we will be looking for optimizations provided by SIMD for parallel computation. For RGB to YCbCr we expect to downsample by taking the average of groups of 4 pixels to determine the RGB value that we will then convert to the YCbCr color space. We will upsample for the inverse operation by interpolating the additional 3 pixels color from the original pixel.

Processor

For our processor we are using an ARM32 emulator to simulate the constraints of this calibre of processor.

We also use a MacOS Tahoe with an Apple Silicon M3 chip and 16GB of RAM for validation testing and iterations before assessing the true performance changes on the ARM32 emulator.

Compiler

We are using GCC to compile our code targeting both our local device(s) for quick confirmation that the code runs, and then targeting the ARM32 microprocessor architecture and copying it to the VM running the emulation software to ensure it runs within the constraints of the architecture.

Optimization Techniques

Our primary optimization will focus on utilizing SIMD to perform parallel operation on our data set. Each pixel is represented by a vector of values that fall within the RGB/YCbCr color spaces. We could be able to parallelize this operation by operating on a 2 by 2 matrix of vectors at once. A concern here will be cache misses. If we are operating on vectors in both row and column order simultaneously we are likely to have cache misses if the size of the image, and therefore the length of a given row, does not fit into memory cache.

To account for the cache misses, optimizations that we're investigating include cache-friendly access by processing images in tiles (16x16) instead of rows to reduce the number of cache misses when the image is larger than the cache. By upsampling and converting our images in tiles, we can take advantage of spatial locality by keeping the values in the cache to reuse them.

Secondary optimizations will include rounding once at the end of our fixed-point arithmetic. Right now, we are rounding after each multiplication in the conversion before adding, but we can

increase accuracy and reduce the number of operations by rounding once at the end. Another optimization will be pivoting from using global variables to local so that instead of having scattered data loads interspersed within our computations, we can have a burst load, then compute and then a burst store. We can also reuse our reads by storing locally since we are currently reloading Y[row][col] multiple times for R, G, B whereas if we replace with local storage we can read once and reuse the values three times. Optimizations like these can make significant gains on the performance of the program since we are cutting memory operations per pixel.

Finally we will be investigating loop unrolling as an optimization to reduce branches and loop overhead. This optimization was already implemented in some areas such as the processing inside the 2x2 blocks, but it will be interesting to compare results between unrolling the column loop by 2 or 4 to see what additional performance we can gain from it.

Additional Information

- Employ Saturating Arithmetic to ensure that we don't overflow, causing a white pixel to become black or vice-versa
- Determine if loop unrolling will provide benefit. It may be necessary in combination with SIMD to ensure more than one row is in memory.
- Converting images before and after conversion to ppm format which is natively viewable on our testing machines to improve iteration time and verification of results.

Project Questions:

- Are we allowed to save or utilize different image formats for easier verification? We are using ppm formats to view them, since the example data we were given isn't directly viewable. Can we convert that to JPG or PNG in our C code to make testing easier?